

APPENDIX

A. Prompt Used

This appendix provides the detailed prompt design used for low-level aspect extraction:

- **Prompt Header:** The prompt header is designed to guide the LLM to have a clearer understanding of the task and to know its role in the task. Here is the prompt that we designed: “You are an experienced annotator, who is performing multi-aspect annotation task based on the answer to a question asked during a mental health survey. You will need to annotate up to 3 most important aspects mentioned among the answer.”
- **Survey question and responses:** The content in this part will be the direct information source that the LLM uses to detect aspects. The question is as follow: “Can you share your thoughts on the mental health or wellness services at your college? Identify aspects that are working well and areas that may need more attention.”. The student responses are varied, but most of which have covered what the question asked.
- **A list of identified aspects:** Since we were calling the LLM API one response at a time, LLM sometimes would generate multiple expressions that have the same meaning for one aspect, which would affect the final result of the aspects. Therefore, we added the following requirement into the prompt: “After identifying the aspects, you should first choose the following pre-defined aspects that have similar meaning with any of aspects that you have identified:”, which is followed by a list that contains the aspects identified in the previous iterations. In this case, the LLM will first consider the aspects that are already identified and choose them, without generating duplicated aspects.

B. Aspect Refinement Details

We first embedded low-level aspects into a high-dimensional space using the pre-trained *SentenceTransformer* model, and then applied *UMAP* to project the representations into two dimensions. We subsequently conducted Silhouette score analysis and obtained an optimal cluster number of 13 (Figure 6). Based on this result, we used K-means to cluster the low-level aspects into 13 groups (Figure 7), where each cluster represents a group of semantically similar aspects. Since some low-level aspects were still assigned to inappropriate groups, we further performed manual validation and refinement based on domain knowledge.

C. High-level Aspects Definition

The following high-level aspect definitions were used by research-trained annotators as the labeling guide during manual annotation. Annotators received training on these definitions and were provided with example excerpts and decision rules to ensure consistent identification of aspect spans and sentiment assignment.

- **On-campus Services:** Emphasize resources that come from the campus.

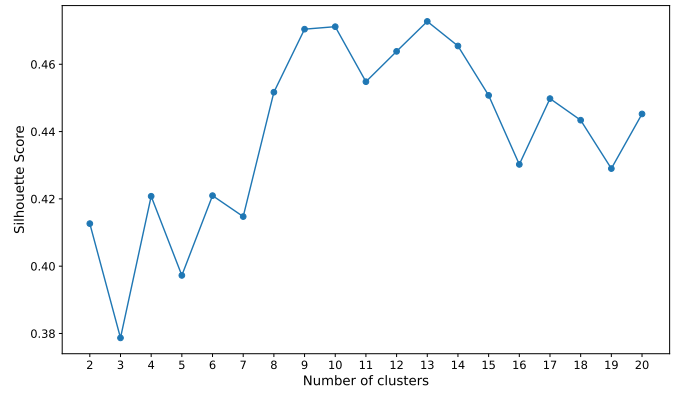


Fig. 6. Silhouette score analysis results on SMILE-College

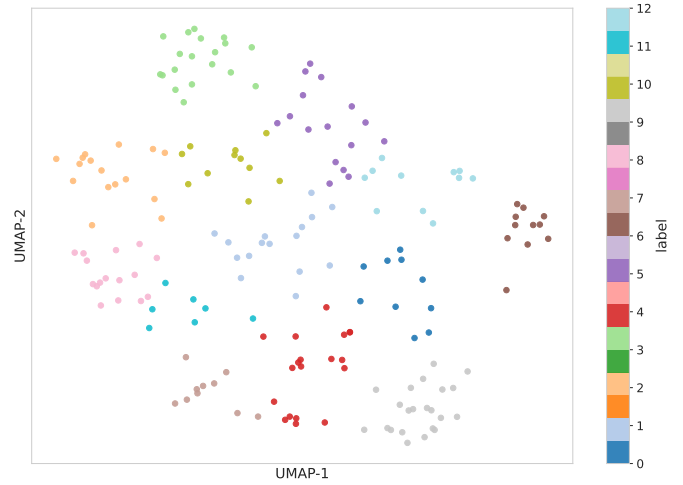


Fig. 7. K-Means clustering results with 13 groups

- **Counseling Service:** Emphasize the counseling process.
- **Mental Health Service:** Emphasize the resources for mental health.
- **Wellness Service:** Emphasize resources for wellness and various support systems.
- **Therapy Service:** Emphasize the therapy process.
- **Hotline Service:** Emphasize the 24 hour and emergency services.
- **Service Availability:** Emphasize the availability/accessibility of the various resources.
- **General:** For those which are not specifically mentioned, label them as general.

D. Dataset distribution analysis

This subsection provides descriptive statistics and visual analyses that summarize the composition and span-level characteristics of AWARE-SMILE. Representative annotated examples are presented in Table I, while the figures referenced below illustrate the aspect-wise sentiment distribution, span

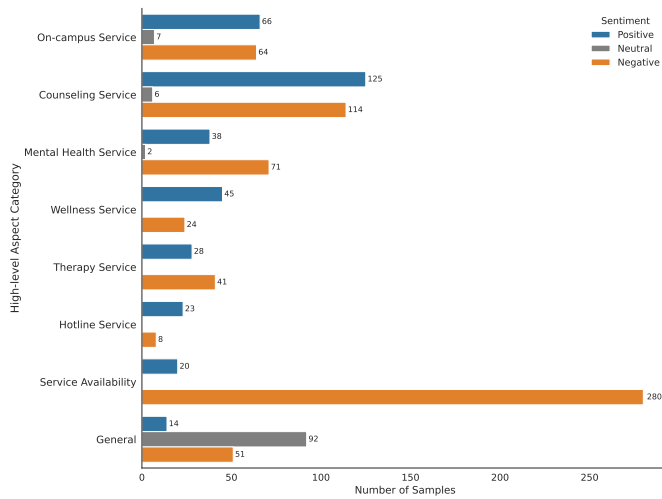


Fig. 8. High-level aspects and sentiment distribution

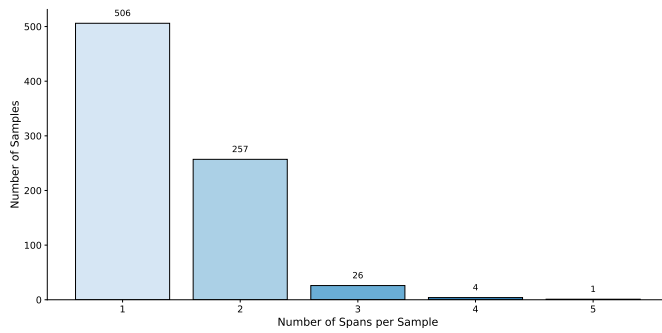


Fig. 9. Span distribution

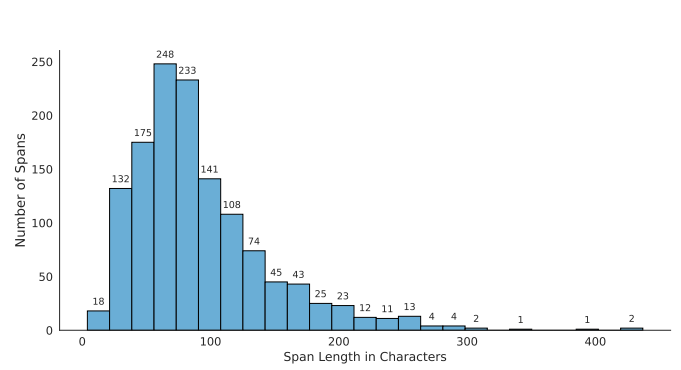


Fig. 10. Span length distribution

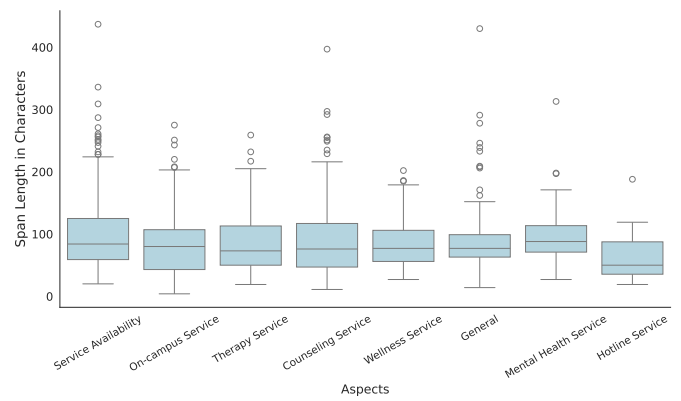


Fig. 11. Span length distribution by aspects

count distribution, span length distribution, and aspect co-occurrence patterns observed in the dataset.

Aspect distribution.: At the aspect level shown in Figure 8, **Service Availability** is the most negatively discussed aspect (228 negative mentions), while **Counseling Service** contains both substantial positive (115) and negative (101) mentions, indicating polarized student experiences. **Mental Health Service** likewise exhibits a predominantly negative distribution (57 negative vs. 28 positive), whereas **Wellness Service** receives comparatively more positive feedback (38 positive vs. 18 negative). In contrast, the **General** category contains the largest proportion of neutral mentions (81), reflecting broadly framed or non-specific comments. Table I further illustrates representative aspect-sentiment annotations and evidence spans extracted from the dataset.

Span distribution.: Figure 9 reports the number of annotated spans per sample. In the annotated corpus, 506 samples contain a single annotated span (63.8%), 257 samples contain two spans (32.4%), 26 samples contain three spans (3.3%), 4 samples contain four spans (0.5%), and 1 sample contains five spans (0.1%). In total, 763 of 794 samples (96.0%) include one or two spans, indicating that the majority of feedback entries focus on a small number of service aspects.

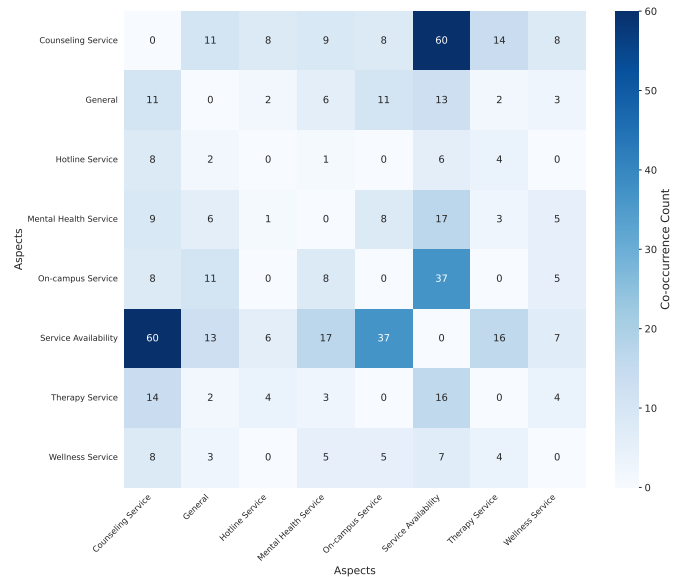


Fig. 12. Aspect co-occurrence matrix

Span length distribution.: Span lengths are concentrated in the 50–125 character range. The histogram in Figure 10 shows the modal bin at 75–100 characters (248 spans), followed by 50–75 characters (233 spans). While most spans are relatively short, a long tail of longer spans is present (including several outliers beyond 300 characters), which likely corresponds to more detailed or compound feedback statements.

Aspect co-occurrence.: Co-occurrence analysis shows that **Service Availability** frequently co-occurs with **Counseling Service** ($n = 60$), **On-campus Service** ($n = 37$), and **Mental Health Service** ($n = 17$), suggesting accessibility concerns often accompany discussions of specific service types. **Service Availability** and **Therapy Service** co-occur in several entries ($n = 16$), while pairs such as **Hotline Service** and **Wellness Service** rarely co-occur, indicating distinct contexts of discussion.

E. Annotation Disagreement Analysis

Following the annotation guidelines, we observed that approximately 20% of samples produced aspect-level sentiment annotations whose aggregated polarity was inconsistent with the original document-level sentiment labels in SMILE-College during the first-pass annotation stage. To improve annotation reliability, these samples were re-examined through an adjudication process among annotators. Samples that still exhibited unresolved inconsistencies after re-review were excluded from the final dataset.

We further analyzed the major sources of disagreement between aspect-level annotations and the original document-level labels. We summarize four primary sources of inconsistency below together with the corresponding adjudication policies used during annotation.

- **Suggestion-related polarity ambiguity**: In the original annotation scheme, labels beyond positive, negative, and neutral (for example, suggestion and limitation) were used in some cases. In particular, suggestion could be interpreted as either constructive/positive or critical/negative, which introduced inconsistency. *Resolution*: treat suggestion as neutral by default unless sentiment-bearing language clearly indicates positive or negative stance.
- **Samples indicating no service exposure**: Some respondents stated that they had never used or never heard of specific services. This created ambiguity between neutral (no direct experience) and negative (possible lack of access/awareness). *Resolution*: label as neutral when no direct evaluative language is present; label negative when the text explicitly complains about lack of access.
- **Mixed sentiment within a single sample**: Some samples expressed both positive and negative views, making it difficult to determine whether the overall polarity should be positive or negative. *Resolution*: retain multi-aspect sentiment labels at span level; for overall dataset-level polarity, adopt a majority/span-weighted aggregation heuristic as described in the Majority Voting subsection.

- **Conditional if-then statements**: Some responses were phrased conditionally (for example, If the service is available, then it is good), which led to different interpretations across annotators. Depending on interpretation, such statements could be labeled as either positive or negative. *Resolution*: annotate conditional statements as neutral unless the conditional clause includes clear evaluative language.

F. Prompt Used

This section provides simplified examples of the prompts used in the LLMs of the proposed framework. All prompts are implemented using structured generation with deterministic decoding (*temperature*=0).

Aspect Proposer Prompt: The Aspect Proposer is responsible for generating and revising candidate aspect categories. It is used in two stages. First, given an input comment and the predefined aspect schema, it selects up to K relevant aspect categories and extracts evidence spans for each candidate. Second, when validator critiques are available, it revises the current candidate set by adding or removing categories according to the feedback. In both stages, the proposer is schema-constrained, and all evidence spans must be copied verbatim from the original input text.

Student feedback / Text x :

[input comment]

Valid aspect schema:

[aspect schema]

Current candidates, if any:

[current candidates]

Critiques from Aspect Validator, if any:

[validator critiques]

Task:

Select or revise up to [top_k] relevant aspect categories. Add or remove categories only from the predefined schema. For each selected category, provide 1–3 evidence substrings copied exactly from Text x .

Rules:

- Do not include categories outside the schema.
- Evidence must be exact substrings from Text x .
- Keep the candidate set small but do not miss clearly relevant categories.
- Return JSON only.

Aspect Validator Prompts: The Aspect Validator contains two prompt-based components: a decision prompt and a critique prompt. The decision prompt verifies whether a proposed aspect category is mentioned in the input text. The critique prompt is used when the decision is uncertain or below the threshold, and generates feedback for revising the candidate aspect set.

Aspect Validator Decision Prompt: The decision prompt takes as input the comment, a target aspect category, the category definition, proposer evidence hints, retrieved positive prototypes, and retrieved hard negatives. The validator decides whether the target aspect is mentioned in the input text and returns evidence spans copied exactly from the original comment.

You are Aspect Validator for aspect category detection (ACD) task. Given text x and category a , decide whether a is mentioned in x . You must provide evidence substrings

copied verbatim from x . If you cannot quote valid evidence from x , you must set `mentioned=false`.

Text x : [input comment]

Current category a : [target category]

Category definition/boundary:

[category definition]

Aspect Proposer evidence hints:

[proposer evidence]

Retrieved positive span prototypes for category c :

[positive prototypes]

Parent texts for prototypes:

[parent texts]

Retrieved hard negatives from other categories:

[hard negatives]

Task: Decide if category a is mentioned in x .

Return JSON only.

Rules:

- `evidence.text` must be a verbatim substring of x , not from prototypes.
- `span` must be character indices into x for that evidence text.
- If no valid evidence is found in x , set `mentioned=false` and `evidence=[]`.
- If the evidence matches a hard negative category better, set `mentioned=false` or lower the score and list `confusable_with`.

Aspect Validator Critique Prompt: The critique prompt is used after the decision step when a candidate aspect is rejected or receives a score below the threshold. It explains why the current candidate may be wrong or uncertain and provides schema-constrained suggestions for revising the candidate set.

You are Aspect Validator for aspect category detection (ACD) task in critic mode. Given text x , target aspect category a , and the current verification result, explain why the category may be wrong or uncertain and suggest edits. You must quote supporting spans verbatim from x with character spans.

Text x : [input comment]

Target category: [target category]

Category definition/boundary:

[category definition]

Current verification result:

[decision result]

Threshold τ for this category: [threshold]

Task: If the category is wrong or uncertain, explain why using spans from x , and suggest how to revise the candidate set.

Return JSON only.

Rules:

- `supporting_spans` must be copied verbatim from x with correct character spans.
- `suggest_add`, `suggest_drop`, and `confusable_with` must use categories from the schema only.
- Keep suggestions minimal.

Sentiment Proposer Prompt: The Sentiment Proposer is used to predict the sentiment polarity toward a given target aspect and extract supporting evidence from the original comment. The prompt takes as input a student feedback comment, one target aspect, and optional diagnostic feedback from previous validation rounds. The model is required to output exactly one sentiment label from NEG, NEU, and POS, together with a confidence score and one to three evidence

spans. All evidence spans must be exact excerpts from the input text.

When previous validation feedback is available, it is appended to the user prompt. This feedback may include problematic evidence spans, reasons for rejection, evidence–aspect relevance warnings, low-confidence signals, and validator-suggested labels. The proposer is then instructed to revise its prediction by selecting different evidence and/or reconsidering the sentiment label.

You are a sentiment classification model for student mental health feedback.

Task: Given one student feedback comment and one target aspect, predict the sentiment toward that target aspect.

Valid sentiment labels: NEG, NEU, POS.

Rules:

- Classify sentiment only for the given target aspect.
- Return exactly one label from NEG, NEU, POS.
- Return JSON only.

Target aspect: [aspect]

Student feedback: [text]

Identify and extract 1–3 key phrases or sentences that support your sentiment classification. These phrases must be exact excerpts from the given text.

[Previous iteration feedback, if any]

Respond only with JSON: {"label": "POS|NEG|NEU", "confidence": 0.0–1.0, "evidence_spans": ["phrase 1", "phrase 2", ...]}

LLM-based Sentiment Consistency Prompt: To compute the LLM-based sentiment support score in SP, the framework additionally uses a lightweight sentiment consistency prompt. Unlike the Sentiment Proposer, this module does not generate evidence spans or perform iterative refinement. Instead, it estimates the sentiment distribution directly from a given evidence phrase and target aspect.

The prompt takes as input an evidence span extracted from the original comment together with the target aspect, and asks the LLM to estimate the probability distribution over the three sentiment labels (POS, NEG, and NEU). The returned probabilities are normalized and used as the LLM-based sentiment consistency signal in the SP validator.

You are a sentiment classifier. Judge the sentiment expressed in the text regarding an aspect.

Text: [evidence]

Aspect: [aspect]

What sentiment does this text express about the aspect?

Return JSON: {"POS": 0.0–1.0, "NEG": 0.0–1.0, "NEU": 0.0–1.0}

The probabilities should approximately sum to 1.0.

G. Aspect Predictor Details

Aspect Proposer: The first component Aspect Proposer generates an initial set of candidate aspect categories from the input text by using a large language model. Formally, given the text x , the proposer produces a candidate set $C^{(0)} = \{(a_i, E_{a_i})\}_{i=1}^m$ where $a_i \in \mathcal{C}$ represents a candidate aspect and E_{a_i} denotes a set of textual evidence spans extracted from the input comment. Each evidence span $e \in E_{a_i}$ must satisfy $e \subset x$, meaning it must be a contiguous substring of the original text. This constraint ensures that all proposed aspects

are grounded in explicit textual evidence rather than inferred from external knowledge. To control computational cost, only the top- K candidate aspects are retained for further validation.

Retrieval-Augmented Memory Bank: To strengthen the verification process, AP incorporates a retrieval-augmented evidence construction mechanism based on a span memory bank. This memory bank is built from domain-related priori knowledge data and consists of annotated aspect spans

$$\mathcal{M}_{\text{ap}} = \{(s_j, a_j)\}_{j=1}^N,$$

where s_j denotes an annotated aspect span associated with aspect a_j , and N is the number of stored memory entities in $\mathcal{M}_{\text{ap}}$. Given an input text x , the framework computes a semantic embedding

$$q = f(x)$$

using a sentence embedding model $f(\cdot)$. The system then retrieves two types of prototype spans. First, **positive prototypes** are retrieved from spans belonging to the same category a :

$$P_a = \text{TopM}_{s_j, a_j=a} \text{sim}(q, f(s_j)).$$

Second, **hard negative prototypes** are retrieved from spans belonging to different categories:

$$N_a = \text{TopN}_{s_j, a_j \neq a} \text{sim}(q, f(s_j)),$$

where $\text{sim}(\cdot, \cdot)$ denotes the embedding similarity function. These retrieved spans provide contextual examples that help the validator distinguish between similar aspect categories.

Aspect Validator: The **Aspect Validator** evaluates whether a candidate aspect is actually mentioned in the text, based on the retrieved positive prototypes P_a and hard negative prototypes N_a . For each candidate aspect a , the validator predicts a decision tuple

$$d_a = (\text{mentioned}_a, S_a, E_a),$$

where E_a represents the supporting evidence spans, $\text{mentioned}_a \in \{0, 1\}$ indicates whether the evidence spans are strictly derived from the input text, and S_a denotes the validation confidence score given by the validator. Each evidence span must correspond to a substring in the original text with character boundaries $\text{span}(e) = [i, j]$ such that $x[i : j] = e$. If no valid evidence can be located in the text, the validator assigns $\text{mentioned}_a = 0$. This evidence-grounded constraint is designed to reduce hallucinated predictions and ensure that aspect identification remains text-faithful.

To further improve robustness, the framework includes a critic-based candidate revision mechanism. When the validator rejects a candidate or assigns a low confidence score $S_a < \tau_{\text{ap}}$, a critic module analyzes the decision and produces diagnostic feedback

$$\Gamma_a = (\text{why_wrong}, \text{confu}, \text{sugg_add}, \text{sugg_drop}),$$

where *why_wrong* explained the reason why the validator to exclude this candidate a . *confu* denotes potentially confused categories, *sugg_add* proposes missing aspect categories, and *sugg_drop* recommends removing incorrect candidates. The

proposer then updates the candidate set based on this feedback through a revision function

$$C^{(t+1)} = \text{Revise}(C^{(t)}, \Gamma^{(t)}).$$

This iterative process continues until the candidate set converges, i.e.,

$$C^{(t+1)} = C^{(t)}$$

or a predefined maximum number of iterations T_{ap} is reached. Finally, the predicted aspect set is obtained as

$$C^* = \{a \mid S_a \geq \tau_{\text{ap}}\}$$

and the module outputs a structured result

$$\mathcal{O}_{\text{ap}} = \{(a, S_a, E_a)\}_{a \in C^*}$$

which includes the detected aspect, their confidence scores, and the supporting textual evidence. Through the combination of candidate generation, retrieval-augmented validation, and iterative self-correction, the AP framework improves both the reliability and interpretability of aspect detection in open-ended student feedback.

H. Sentiment Predictor Details

Sentiment Proposer: For each aspect a and comment x , the proposer uses a large language model to infer a candidate sentiment label, a confidence score, and a set of supporting evidence spans. Formally, the proposer outputs:

$$p_a^{(t)} = (y_a^{(t)}, r_a^{(t)}, E_a^{(t)}),$$

where $y_a^{(t)} \in \mathcal{Y}$ is the proposed sentiment label at iteration t , $r_a^{(t)} \in [0, 1]$ is the proposer confidence, and $E_a^{(t)} = \{e_1, \dots, e_m\}$ is a set of extracted evidence spans. Each evidence span must be an exact excerpt from the original text, rather than a paraphrase, so that the sentiment decision remains explicitly grounded in the input comment. In practice, the proposer is prompted to classify sentiment with respect to the target aspect and extract one to three phrases or sentences that directly support the predicted polarity.

To support validation across instances, SP maintains a separate retrieval-augmented memory bank of previously accepted aspect-sentiment evidence patterns. The memory can be formalized as:

$$\mathcal{M}_{\text{sp}} = \{(a_i, y_i, E_i, \gamma_i)\}_{i=1}^N,$$

where a_i denotes an aspect, $y_i \in \mathcal{Y}$ is the stored sentiment label, E_i is the associated evidence phrase set, and γ_i is the confidence score of that memory entry. Each entry can additionally be associated with an embedding vector computed from the concatenated evidence text. This memory allows the validator to compare newly proposed evidence against previously validated sentiment patterns for the same aspect. As a result, validation is not performed solely on the current instance, but also leverages accumulated experience from earlier examples.

Sentiment Validator: The Sentiment Validator evaluates whether the proposed label is well supported by the evidence. Given a candidate prediction $P_a^{(t)}$, the validator computes a validation score $S_a^{(t)} \in [0, 1]$, which measures how reliably the current evidence supports the proposed sentiment for aspect a . The validation mechanism combines two complementary signals. The first is a **memory-based similarity signal**, which compares the proposed evidence against memorized evidence patterns of the same aspect. Let q denote the embedding of a query evidence phrase and let $\mathcal{M}_a \subset \mathcal{M}_{\text{sp}}$ be the subset of memory entries associated with aspect a . For each query span, the validator retrieves the top- K most similar memory entries and accumulates similarity support for each sentiment label. If we denote by $\text{sim}(q, m)$ the cosine similarity between the query span and a memory entry m , then the support score for label y is aggregated as

$$\text{score}_{\text{mem}}(y) = \frac{1}{|\mathcal{N}_y| + \epsilon} \sum_{m \in \mathcal{N}_y} \text{sim}(q, m) \cdot S_{\text{rel}}(q, a),$$

where $\mathcal{N}_y$ is the set of retrieved memory entries with label y , and $S_{\text{rel}}(q, a)$ measures how relevant the evidence phrase is to the description of target aspect by calculating the cosine similarity between q and a . This relevance term prevents the validator from over-trusting phrases that are sentiment-bearing but semantically unrelated to the aspect under evaluation.

The second signal is an **LLM-based sentiment consistency signal**. Instead of relying only on retrieved memory, the validator also uses a large language model to estimate the sentiment distribution expressed by the evidence phrase itself with respect to aspect a . This yields a probability vector

$$\mathbf{p}_{\text{llm}}(q, a) = (p_{\text{POS}}, p_{\text{NEG}}, p_{\text{NEU}}),$$

where the three probabilities sum approximately to one. For each candidate label y , the model computes a margin-based support score relative to the strongest competing label:

$$\text{supp}_{\text{llm}}(y) = \text{clip}\left(p_y - \max_{y' \neq y} p_{y'}, -1, 1\right).$$

Similarly, the memory-based signal $\text{score}_{\text{mem}}(y)$ is converted into a label-level margin score by comparing the support of the proposed label against the best alternative, denoted as $\text{supp}_{\text{sim}}(y)$. In this way, both signals are expressed on a common scale and can be fused consistently. The final validation support combines both signals:

$$\text{supp}(y) = \alpha \cdot \text{supp}_{\text{sim}}(y) + (1 - \alpha) \cdot \text{supp}_{\text{llm}}(y),$$

where $\alpha \in [0, 1]$ controls the balance between retrieval-based similarity support and the LLM-based sentiment consistency signal. For the proposed label $y_a^{(t)}$, the final validation score is mapped into $[0, 1]$ as

$$S_a^{(t)} = \frac{\text{supp}\left(y_a^{(t)}\right) + 1}{2}.$$

A higher value of $S_a^{(t)}$ indicates that the evidence is both semantically aligned with the aspect and sentimentally consistent with the proposed polarity. This score is used as the acceptance criterion in the iterative loop.

An important feature of SP is its cold-start strategy. In early stages, the memory bank may contain no validated examples for a given aspect. In that case, the validator cannot rely on retrieval support and instead estimates the validation score using only aspect relevance and sentiment consistency. Let S_{rel} denote the relevance between the evidence and the aspect description, and let S_{llm} denote the normalized sentiment margin estimated by the LLM. The cold-start validation score is computed as

$$S_a^{(t)} = \sigma(\beta S_{\text{rel}} + \gamma S_{\text{llm}} + b),$$

where $\sigma(\cdot)$ is the sigmoid function and β, γ, b are hyperparameters. This mechanism allows the system to bootstrap sentiment memory even before sufficient historical entries have been accumulated.

Similar to AP, the proposer and validator in SP are connected through an iterative refinement loop. At iteration t , the proposer generates a candidate prediction $P_a^{(t)}$, and the validator computes its validation score $S_a^{(t)}$. The prediction is accepted if $S_a^{(t)} \geq \tau_{\text{sp}}$, where τ_{sp} is a predefined sentiment acceptance threshold. If the score falls below the threshold, the validator returns diagnostic feedback, including why the evidence is insufficient, which sentiment labels are potentially confused, and which evidence spans should be revised or replaced. The feedback is then passed back to the proposer in the next iteration so that it can select different evidence spans or revise the sentiment label. The process continues for at most T_{sp} rounds, if no candidate passes the gate after all iterations, the framework outputs the best-scoring candidate while marking it as *uncertain*.

The final output of SP for each detected aspect a is a structured sentiment prediction:

$$o_a = (\hat{y}_a, \hat{E}_a, \hat{S}_a),$$

where $\hat{y}_a$ is the accepted sentiment label, $\hat{E}_a$ is the set of supporting evidence spans, and $\hat{S}_a$ is the final validator score. Once a prediction is accepted, its aspect, sentiment label, and evidence phrases are written into the memory bank $\mathcal{M}_{\text{sp}}$ as a new memory entry. This makes the framework self-improving across samples, since future predictions can benefit from a richer repository of validated sentiment evidence.

I. Hyperparameter Settings

For AP, the aspect proposer generates up to $K = 8$ candidate aspect categories for each input text. Each candidate is associated with a confidence score and a set of extracted evidence spans. During validation, the system retrieves top- $M = 5$ positive prototypes from the same category and top- $N = 3$ hard negative prototypes from other categories. Candidates with validation scores above $\tau_{\text{ap}} = 0.4$ are accepted. The iterative proposer-validator process is executed for at most $T_{\text{ap}} = 2$ rounds.

For SP, sentiment prediction is performed independently for each detected aspect. For each evidence span, the validator retrieves top- $K_m = 5$ similar memory entries from the same aspect category and aggregates label-wise support scores based on embedding similarity. In parallel, an LLM-based consistency module estimates sentiment probabilities directly from the evidence spans. The two signals are combined using a weighted coefficient $\alpha = 0.5$. Predictions with scores above $\tau_{sp} = 0.5$ are accepted, while rejected predictions are iteratively refined for at most $T_{sp} = 3$ rounds.

J. LLM Backbone Settings

We employ a unified LLM backbone for both proposer and validator components in AP and SP. All modules are implemented through API-based inference with deterministic decoding settings (*temperature=0*). Prompts are designed using schema-constrained generation to enforce valid aspect categories, sentiment labels, and exact evidence extraction.

The proposer generates grounded predictions together with evidence spans, while the validator separately performs decision verification and critique generation. Additional post-processing is applied to ensure exact evidence span matching within the original text.

No task-specific fine-tuning is performed. The same prompting framework is consistently applied across all experiments.

We additionally conduct ablation studies by replacing the default backbone with lightweight LLMs, including *gpt-4o-mini*, *claude-haiku-4.5*, and *gemini-2.5-flash*. All models are evaluated under identical prompting and decoding configurations.

K. Training and Hardware

All supervised baselines are implemented using PyTorch and HuggingFace Transformers. Experiments are conducted on a single NVIDIA A6000 GPU with 48GB memory and CUDA support.